



UNIVERSIDADE DA CORUÑA

Facultade de Informática

Máster Universitario en Enxeñaría en Informática

TRABALLO FIN DE MÁSTER

Aplicación para la Gestión de Emergencias

Estudiante: Adrián García Vázquez

Dirección: Manuel Álvarez Díaz

Monforte de Lemos, mayo de 2026.

Dedicatoria

Agradecimientos

Gracias a Manuel, mi tutor, por su paciencia infinita y sus consejos que me han permitido hacer realidad este proyecto.

Gracias a mi familia, mis amigos y a mi novia por el apoyo incondicional y ánimos.

También agradezco a mis compañeros y profesorado de MUEI que me han ayudado a mejorar como informático y como persona.

Resumen

Galicia afronta de forma recurrente emergencias de alto impacto (incendios, inundaciones, derrumbes y riesgos industriales), que exigen una respuesta rápida y coordinada. En este contexto, la gestión operativa se ve limitada por la fragmentación de herramientas, la escasa integración de datos y la necesidad de coordinar a múltiples actores en escenarios de alta presión.

Este Trabajo de Fin de Máster parte de un sistema previo centrado en incendios forestales y lo amplía hacia una plataforma de coordinación multi-riesgo para Galicia. La solución se apoya en dos pilares: una plataforma web evolucionada y una aplicación móvil nativa en Kotlin para uso en campo y participación ciudadana. El sistema centraliza alertas y emergencias, integra geolocalización y reportes multimedia, coordina recursos y ofrece visualización cartográfica en tiempo real de eventos y despliegues. Además, incorpora notificaciones push y un histórico para análisis post-emergencia, con indicadores de tiempos de respuesta y recursos utilizados. Con ello, el proyecto mejora la interoperabilidad, optimiza la toma de decisiones y refuerza la capacidad de respuesta en el territorio gallego.

Abstract

Galicia recurrently faces high-impact emergencies (fires, floods, landslides, and industrial hazards) that require fast, coordinated response. In this context, operational management is limited by fragmented tools, weak data integration, and the need to coordinate multiple stakeholders in high-pressure scenarios.

This Master's Thesis extends a previous wildfire-focused system into a multi-risk coordination platform for Galicia. The solution is built on two pillars: an enhanced web platform and a native Kotlin mobile app for field use and citizen participation. The system centralizes alerts and incidents, integrates geolocation and multimedia reporting, supports resource coordination, and provides real-time map visualization of events and deployments. It also includes push notifications and historical records for post-incident analysis, with indicators on response times and resource usage. Overall, the project improves interoperability, supports better decision-making, and strengthens emergency response capacity across the Galician territory.

Palabras clave:

- Galicia
- Gestión de emergencias
- Geolocalización
- Coordinación de personas y recursos
- Aplicación Web
- Java - Spring Boot
- React - Redux
- Aplicación móvil Kotlin

Keywords:

- Galicia
- Emergency management
- Geolocation
- Aplicación Web
- Coordination of people and resources
- Java
- React - Redux
- Kotlin mobile application

Índice general

1	Introducción	1
1.1	Determinación de la situación actual	1
1.2	Alcance y objetivos	2
1.2.1	Objetivos principales	3
1.2.2	Objetivos concretos	3
1.3	Estructura de la memoria	4
2	Base Tecnológica	5
2.1	Lenguajes	5
2.1.1	Java	5
2.1.2	Kotlin	5
2.1.3	SQL	5
2.1.4	HTML Y CSS	5
2.1.5	JavaScript	6
2.1.6	JSX	6
2.1.7	LaTeX	6
2.1.8	JSON	6
2.2	Frameworks y librerías servidor	7
2.2.1	Spring	7
2.2.2	SpringBoot	7
2.2.3	SpringData	7
2.2.4	JPA	7
2.2.5	Hibernate	7
2.2.6	JDBC	7
2.3	Frameworks y librerías web	8
2.3.1	React	8
2.3.2	Redux	8

2.3.3	RTKQuery	8
2.3.4	MaterialUI	8
2.3.5	Open Weather Map	8
2.3.6	MapLibre	8
2.4	Frameworks y librerías web	9
2.4.1	Gradle	9
2.4.2	JetPack Compose	9
2.4.3	Material Design	9
2.4.4	HttpClient	9
2.4.5	Firebase Cloud Messaging	9
2.4.6	FirebaseMessagingService	9
2.5	Pruebas	10
2.5.1	JUnit	10
2.6	Protocolos	10
2.6.1	HTTP/S	10
2.6.2	REST	10
2.7	Herramientas de desarrollo	10
2.7.1	IntelliJ	10
2.7.2	Postman	10
2.7.3	VSCode	11
2.7.4	DBeaver	11
2.7.5	GIT	11
2.7.6	Apache Maven	11
2.7.7	Create React App	11
2.8	Sistemas de gestión de base de datos	12
2.8.1	PostgreSQL	12
2.8.2	PostGIS	12
2.9	Tecnologías de alojamiento de servicios	12
2.9.1	Tomcat	12
2.9.2	Webpack	12
2.9.3	Serve	12
2.9.4	Docker	12
3	Estado del arte	14
3.1	Aplicaciones de gestión de incidencias	14
3.1.1	Jira	14
3.1.2	Asana	15
3.1.3	Línea Verde	16

3.2	Aplicaciones sobre geolocalización de información	17
3.2.1	Global Forest Watch - Fires	17
3.2.2	InciWeb	18
3.2.3	Google Maps	19
3.3	Aplicaciones orientadas a organizar grupos de trabajo	19
3.3.1	Microsoft Teams	19
3.3.2	Odoo	20
3.3.3	PCAM Gestión	20
3.3.4	XeoCode Lite	21
3.4	Iniciativas públicas recientes	21
3.4.1	Xunta de Galicia: nueva app y ampliación de detección temprana . . .	21
3.5	Conclusión	21
4	Análisis de viabilidad	23
4.1	Viabilidad temporal	23
4.2	Viabilidad tecnológica	23
4.3	Viabilidad económica	24
4.4	Conclusión del análisis de viabilidad	24
5	Introducción al desarrollo realizado	25
5.1	Introducción	25
5.2	Arquitectura de la Solución	25
5.3	Metodología e iteraciones	25
6	Planificación y análisis de costes	27
6.1	Planificación	27
6.2	Sprints	29
6.2.1	Sprint 0: Contextualización	29
6.2.2	Sprint 1: Esqueleto y arquitectura base de aplicación móvil	29
6.2.3	Sprint 2: Actualizar la organización de recursos	29
6.2.4	Sprint 3: Emergencias y asignaciones básicas	29
6.2.5	Sprint 4: Nuevo sistema de logs operativos	29
6.2.6	Sprint 5: Notificaciones en Android y aceptación de asignaciones . . .	30
6.2.7	Sprint 6: Protocolos semiautomáticos (motor de reglas simple)	30
6.2.8	Sprint 7 y 8: Finalización, pruebas y despliegue	30
6.3	Seguimiento	30
6.4	Análisis de costes	30
6.4.1	Estimación del esfuerzo	31

6.4.2	Costes directos de personal	31
6.4.3	Costes indirectos	31
6.4.4	Resumen económico	32
6.4.5	Conclusión del análisis de costes	32
7	Requisitos del sistema	33
7.1	Introducción	33
7.2	Actores	33
7.3	Casos de uso	33
7.4	Modelo de casos de uso	33
7.5	Prototipado de interfaz	34
8	Diseño	35
8.1	Introducción y objetivos	35
8.2	Resumen de patrones usados	35
8.3	Arquitectura general	35
8.4	Subsistema Backend	35
8.4.1	Objetivos	35
8.4.2	Arquitectura	35
8.4.3	Modelo del dominio	36
8.4.4	Capa de acceso a datos	36
8.4.5	Capa de lógica de negocio	37
8.4.6	Capa servicios	37
8.4.7	Otros?	37
8.5	Subsistema Frontend	37
8.5.1	Objetivos	37
8.5.2	Arquitectura	37
8.5.3	Capa de acceso al servicio	38
8.5.4	Capa de componentes de interfaz	38
8.5.5	Otros?	38
9	Implementación	39
9.1	Software requerido	39
9.2	Estructura	39
9.3	Instrucciones de compilación	39
10	Pruebas	40
10.1	Introducción	40
10.2	Pruebas unitarias	40

10.3 Pruebas de integración	40
10.4	41
11 Conclusiones y futuras líneas de trabajo	42
11.1 Conclusiones	42
11.2 Aprendizajes	42
11.3 Futuras líneas de trabajo	42
A Material adicional	44
A.1 Instalación del Software	44
A.2 Manual de usuario	44
Lista de acrónimos	45
Bibliografía	47

Índice de figuras

3.1	Vista de proyecto en Jira	14
3.2	Vista de proyecto en Asana	15
3.3	Vista de la app Línea Verde	16
3.4	Ejemplo de mapa de detección de incendios (AFIS / capa de incendios)	17
3.5	Vista de la app InciWeb	18
3.6	Ejemplo de capa/alertas en Google Maps	19
3.7	Vista equipos en Microsoft Teams	19
3.8	Vista equipos en Odoo	20
3.9	Panel de control en PCAM Gestión	20

Índice de tablas

3.1	Tabla de comparativa entre aplicaciones	22
6.1	Planificación del Sprint 3: Emergencias y asignaciones básicas	28
6.2	Desglose de costes estimados del proyecto	32

Introducción

GALICIA presenta una elevada exposición a emergencias en su territorio, entre las que destacan especialmente los incendios forestales, las inundaciones derivadas de episodios de lluvia intensa, los derrumbes de carreteras o terraplenes y determinados riesgos industriales. En los últimos años han surgido emergencias que ilustran su importancia y su impacto operativo: campañas con incendios que han afectado a superficies significativas según la estadística oficial [1]; episodios de abril de 2026 que motivaron una decena de incidencias en Ourense y la activación de alertas en las Rías Baixas [2, 3]; y movimientos sísmicos locales perceptibles por la población, como el registrado en Ponteceso en 2026 [4]. La complejidad operativa de la respuesta exige coordinación entre múltiples actores, toma de decisiones en ventanas temporales reducidas y disponibilidad de información operativa fiable. Estos retos son tanto tecnológicos como organizativos y normativos, ya que confluyen distintos niveles administrativos y tecnologías

Este [Trabajo Fin de Máster \(TFM\)](#) se apoya y amplía el trabajo previo desarrollado en el [Trabajo Fin de Grado \(TFG\)](#) "Aplicación para coordinación de equipos para la prevención y extinción de incendios forestales" [5], que documenta el diseño e implementación de una aplicación web destinada a centralizar avisos, recursos y despliegues en el contexto de incendios forestales en Galicia. El [TFG](#) aporta decisiones y soluciones de diseño que se toman como punto de partida en este [TFM](#).

1.1 Determinación de la situación actual

La situación actual combina capacidades consolidadas de detección, alerta y despliegue con limitaciones de integración entre sistemas y organizaciones. En el ámbito forestal, existen estadísticas nacionales y autonómicas que permiten caracterizar la magnitud del problema [1]. Galicia dispone además de una estructura operativa asentada en torno a [Axencia Galega de Emerxencias \(112 Galicia\)](#) ([AXEGA](#)) y cuenta con planes de emergencia por tipología de riesgo

a nivel autonómico [6, 7].

Galicia dispone de una organización y planificación detallada para distintas tipologías de riesgo —por ejemplo, [Plan de Prevención y Defensa Contra los Incendios Forestales de Galicia \(PLADIGA\)](#) para riesgos forestales—. La Consellería de Presidencia centraliza catálogos, protocolos y guías municipales; la Consellería de Medio Rural publica documentación técnica y memorias sobre defensa del monte; y [AXEGA](#) mantiene boletines e instrumentos operativos. Sin embargo, estos planes y protocolos están dispersos entre distintos organismos y repositorios, y no están unificados en una solución digital compartida y accesible tanto para los equipos de gestión de emergencias como para la ciudadanía común [6, 7, 8].

No obstante, persisten dificultades operativas que limitan la eficacia conjunta:

- **Fragmentación de la información:** avisos ciudadanos, información meteorológicas, mapas de emergencias y estado de recursos residen en sistemas segregados.
- **Heterogeneidad organizativa:** organismos y equipos tienen procedimientos y necesidades distintas (coordinación estratégica, mando táctico, intervención en campo).
- **Variabilidad del riesgo y concurrencia de eventos:** no es realista el tratamiento aislado de las emergencias, ciertas emergencias suelen estar relacionadas con otras emergencias, exigiendo un modelo multi-riesgo [3, 7].
- **Presión temporal y trazabilidad:** la necesidad de decisiones rápidas hace imprescindible la actualización en tiempo real y el registro de actuaciones para análisis posteriores.

El análisis realizado en el [TFG](#) muestra que estas limitaciones no son debidas a la ausencia de medios, sino a falta de cohesión operativa digital. Por ello, este [TFM](#) propone una evolución del sistema desarrollado en el [TFG](#) hacia una plataforma de coordinación que mejore la integración, la trazabilidad y el soporte a la decisión en contextos multi-riesgo.

1.2 Alcance y objetivos

El alcance de este [TFM](#) es evolucionar la solución técnica y los modelos operativos desarrollados en el [TFG](#) hacia una plataforma de coordinación de emergencias multi-riesgo para Galicia, con dos componentes principales: una plataforma web para gestión y coordinación, y una aplicación móvil nativa para uso en campo y participación ciudadana. Este [TFM](#) parte explícitamente del trabajo técnico y de campo documentado en el [TFG](#) [5].

La solución propuesta se alinea con el trabajo realizado por protección civil y otros organismos de respuesta a emergencias, priorizando la interoperabilidad entre estos organismos, sin pretender sustituir sistemas institucionales existentes, sino complementarlos aportando una capa unificada de gestión, visualización y soporte a la decisión [9, 10].

1.2.1 Objetivos principales

A continuación se enumeran los principales objetivos que debe cumplir la aplicación a desarrollar en este trabajo:

1. Extender el dominio funcional del TFG hacia un enfoque multi-riesgo (incendios, inundaciones, derrumbes y riesgos industriales) acorde con los planes autonómicos.
2. Ampliar funcionalidades de la plataforma web existente que centraliza la gestión de alertas, emergencias, recursos y despliegues operativos.
3. Diseñar e implementar una aplicación móvil nativa en Kotlin para reporte ciudadano y soporte a la intervención en campo.
4. Mejorar la coordinación multi-organizativa, integrando en un mismo flujo de trabajo a coordinadores, brigadas, protección civil y ciudadanía.
5. Reforzar la trazabilidad y el análisis post-emergencia mediante históricos e indicadores operativos.

1.2.2 Objetivos concretos

1. Contextualización: esclarecer el valor aportado por el TFM y definir el nuevo dominio multi-riesgo.
2. Definir la arquitectura de la aplicación móvil y entregar un esqueleto que permita añadir funcionalidades progresivamente.
3. Introducir un modelo unificado de recursos y organizaciones y definir sus capacidades y área operativa.
4. Permitir la creación y edición de emergencias (por punto o por cuadrante) y la creación o propuesta manual de asignaciones.
5. Introducir un registro de logs operativos de las emergencias y de las asignaciones de recursos que permita el análisis posterior de las emergencias y la generación de indicadores.
6. Integrar notificaciones móviles mediante [Firebase Cloud Messaging \(FCM\)](#), registrar dispositivos y definir el flujo de notificación y aceptación de asignaciones.
7. Implementar el almacenamiento de protocolos (por ejemplo, campo [JavaScript Object Notation Binary \(JSONB\)](#)) y un motor de reglas capaz de generar propuestas de asignación automáticamente.

1.3 Estructura de la memoria

La memoria se organiza en once capítulos y anexos. A continuación se ofrece un resumen del contenido de cada capítulo:

1. Capítulo 1: Presenta el contexto general, describe la situación actual en Galicia, expone las motivaciones del trabajo y enuncia los objetivos generales y concretos del proyecto.
2. Capítulo 2 (2): Detalla la base tecnológica empleada: componentes de backend y frontend, bases de datos, servicios y librerías relevantes, y las decisiones arquitectónicas que sustentan la implementación.
3. Capítulo 3 (3): Revisa el estado del arte, compara soluciones existentes y extrae lecciones aplicables al diseño de la plataforma propuesta.
4. Capítulo 4 (4): Analiza la viabilidad del proyecto desde las perspectivas técnica, organizativa y normativa, e incluye una estimación de recursos y riesgos.
5. Capítulo 5 (5): Documenta el desarrollo realizado, describiendo la arquitectura global del sistema, los módulos principales y las integraciones implementadas.
6. Capítulo 6 (6): Presenta la planificación temporal del trabajo, los hitos y la metodología seguida para organizar las iteraciones de desarrollo.
7. Capítulo 7 (7): Especifica los requisitos funcionales y no funcionales que guían el diseño y la implementación de la plataforma.
8. Capítulo 8 (8): Describe el diseño funcional y técnico, incluyendo el modelo de datos, las APIs, los flujos de interacción y las decisiones de UX/UI.
9. Capítulo 9 (9): Detalla la implementación: estructura de código, despliegue, configuraciones y ejemplos de componentes desarrollados.
10. Capítulo 10 (10): Documenta las pruebas realizadas (unitarias, de integración y de aceptación), su cobertura y los resultados obtenidos.
11. Capítulo 11 (11): Resume las conclusiones, las aportaciones del trabajo y propone líneas futuras de mejora y continuidad.

Capítulo 2

Base Tecnológica

A continuación se muestra una posible agrupación de las tecnologías usadas junto con una breve descripción de cada una de ellas.

2.1 Lenguajes

2.1.1 Java

Java [11] es un lenguaje de programación orientado a objetos, que se caracteriza por su portabilidad, seguridad y facilidad de uso. Se utiliza ampliamente en el desarrollo de aplicaciones empresariales, juegos, aplicaciones móviles y aplicaciones web, entre otros.

2.1.2 Kotlin

Kotlin [12] es un lenguaje de programación moderno y conciso que se ejecuta en la máquina virtual de Java. Se ha utilizado en este proyecto para la aplicación Android debido a su interoperabilidad con Java y su sintaxis más limpia y expresiva.

2.1.3 SQL

Structured Query Language (SQL)[13] es un lenguaje de consulta estructurado utilizado para administrar y manipular bases de datos relacionales. Se utiliza para realizar tareas como la inserción, actualización y eliminación de datos, así como para la generación de informes y análisis de datos. Es uno de los lenguajes más utilizados en el mundo de la informática y es esencial para la administración y gestión de datos.

2.1.4 HTML Y CSS

HyperText Markup Language (HTML) y Cascading Style Sheet (CSS) [14] son lenguajes utilizados en la creación de páginas web. HTML se utiliza para estructurar el contenido de la

página, mientras que [CSS](#) se utiliza para darle estilo y presentación. [HTML](#) utiliza una serie de etiquetas para definir la estructura de la página, como títulos, encabezados, párrafos y enlaces, entre otros. [CSS](#) se utiliza para dar formato y estilo a la página como colores, fuentes, márgenes y bordes.

2.1.5 JavaScript

JavaScript [15] es un lenguaje de programación dinámico y no fuertemente tipado, utilizado principalmente en el desarrollo de aplicaciones web interactivas y dinámicas. Se utiliza para agregar funcionalidad a las páginas web, como animaciones, validación de formularios y efectos visuales.

2.1.6 JSX

JSX [16] es una extensión de sintaxis utilizada en la biblioteca de JavaScript React. JSX permite a los desarrolladores escribir código JavaScript que se parece mucho al marcado HTML. Esta sintaxis especial facilita la creación de componentes de interfaz de usuario reutilizables en React. Cuando se compila, JSX se traduce a llamadas regulares de JavaScript que manipulan elementos virtuales.

2.1.7 LaTeX

LaTeX [17] es un sistema de composición de documentos ampliamente utilizado en el ámbito académico y científico. Con LaTeX, los usuarios pueden crear documentos de alta calidad con una estructura lógica y una apariencia profesional.

2.1.8 JSON

[JavaScript Object Notation \(JSON\)](#) [18] es un formato de intercambio de datos textual, ligero y ampliamente utilizado. Se basa en una estructura de pares clave-valor y se utiliza para representar información estructurada de manera legible tanto para humanos como para máquinas. JSON es especialmente popular en el desarrollo de aplicaciones web, donde se utiliza para transmitir datos. Su sintaxis simple y su facilidad de uso lo convierten en una opción común para el intercambio de datos en muchas aplicaciones modernas.

2.2 Frameworks y librerías servidor

2.2.1 Spring

Spring [19] es un framework de desarrollo de aplicaciones de código abierto para Java. Proporciona una amplia gama de características y funcionalidades que facilitan el desarrollo de aplicaciones empresariales robustas y escalables. Spring se basa en el patrón de diseño de [Inversión de Control \(IoC\)](#) y el contenedor de [IoC](#) de Spring gestiona la creación y administración de objetos en una aplicación. Además del contenedor de [IoC](#), Spring ofrece módulos para la integración con bases de datos, servicios web, seguridad, transacciones y más.

2.2.2 SpringBoot

Spring Boot [20] es un framework para desarrollar aplicaciones en Java con Spring. Proporciona una configuración predeterminada que puede ser personalizada y está diseñado para simplificar el desarrollo de aplicaciones Spring al eliminar gran parte de la complejidad relacionada con la configuración.

2.2.3 SpringData

Spring Data [21] es un framework que simplifica el acceso a datos desde aplicaciones Java con bases de datos relacionales y no relacionales. Spring Data reduce la cantidad de código necesario para implementar el acceso a datos y proporciona una abstracción para trabajar con diferentes tipos de bases de datos.

2.2.4 JPA

[Java Persistence API \(JPA\)](#) [22] proporciona una capa de abstracción para trabajar con bases de datos relacionales en Java, permitiendo a los desarrolladores interactuar con las bases de datos utilizando objetos en lugar de [SQL](#).

2.2.5 Hibernate

Hibernate [23] proporciona una solución de mapeo objeto-relacional para Java que permite a los desarrolladores interactuar con las bases de datos utilizando objetos en lugar de [SQL](#). Hibernate implementa la especificación [JPA](#).

2.2.6 JDBC

[Java Database Connectivity \(JDBC\)](#) [24] es una [Application Programming Interface \(API\)](#) de Java que proporciona un conjunto de clases e interfaces para interactuar con bases de datos

relacionales. JDBC facilita la conexión, consulta y manipulación de datos en bases de datos a través del lenguaje de programación Java.

2.3 Frameworks y librerías web

2.3.1 React

React [25] es una biblioteca de JavaScript para construir interfaces de usuario. Proporciona una forma declarativa para construir componentes de interfaz de usuario reutilizables y altamente modulares.

2.3.2 Redux

Redux [26] es una biblioteca de JavaScript para manejar el estado de las aplicaciones. Proporciona una forma de manejar el estado de las aplicaciones, haciendo que la gestión del estado sea más fácil de entender y depurar.

2.3.3 RTKQuery

RTKQuery [27] es una biblioteca de JavaScript para manejar las solicitudes a la [API](#) en aplicaciones Redux. Proporciona una solución escalable y fácil de usar, simplificando la gestión del estado de la aplicación en respuesta a las solicitudes de la [API](#).

2.3.4 MaterialUI

Material [28] es un framework de diseño de componentes de React basado en los principios de diseño de Material Design. Proporciona una amplia variedad de componentes preconstruidos y personalizables, lo que facilita la creación de interfaces de usuario vistosas y funcionales.

2.3.5 Open Weather Map

Open Weather Map [29] es un servicio en línea que proporciona datos meteorológicos y de pronóstico en tiempo real. Permite a los desarrolladores acceder a una amplia gama de información climática, como temperatura, humedad, velocidad del viento, condiciones atmosféricas, pronósticos a corto y largo plazo, entre otros.

2.3.6 MapLibre

MapLibre [30] es un proyecto libre que proporciona funcionalidades para la visualización de mapas en 2D y 3D, la capacidad de agregar marcadores y rutas, la personalización de estilos y la integración con datos geoespaciales.

2.4 Frameworks y librerías web

2.4.1 Gradle

Gradle [31] es un sistema de automatización de compilación de código abierto que se utiliza para construir, probar y desplegar aplicaciones. Gradle es altamente flexible y se puede utilizar para proyectos de cualquier tamaño, desde pequeñas aplicaciones hasta grandes sistemas empresariales.

2.4.2 JetPack Compose

Jetpack Compose (Compose) [32] permite a los desarrolladores crear interfaces de usuario de forma declarativa utilizando Kotlin. Proporciona una forma más sencilla y eficiente de construir interfaces de usuario en Android, eliminando la necesidad de escribir código XML para diseñar las pantallas.

2.4.3 Material Design

Material Design (Material) [33] es un sistema de diseño desarrollado por Google que proporciona un conjunto de principios y directrices para crear interfaces de usuario atractivas, intuitivas y consistentes en diferentes plataformas y dispositivos.

2.4.4 HttpClient

HttpClient es una biblioteca de Kotlin que proporciona una forma sencilla y eficiente de realizar solicitudes HTTP. Permite a los desarrolladores enviar solicitudes HTTP, manejar respuestas y gestionar conexiones de red de manera fácil y flexible.

2.4.5 Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) [34] es un servicio de mensajería en la nube proporcionado por Google que permite a los desarrolladores enviar notificaciones y mensajes a dispositivos móviles y aplicaciones web.

2.4.6 FirebaseMessagingService

FirebaseMessagingService (FMS) [35] es una clase de servicio en Android que se utiliza para manejar la recepción de mensajes y notificaciones enviados a través de FCM.

2.5 Pruebas

2.5.1 JUnit

JUnit [36] es un framework de pruebas unitarias para el lenguaje de programación Java. Proporciona un conjunto de anotaciones y aserciones para facilitar la creación y ejecución de pruebas unitarias en código Java. JUnit se ha convertido en una de las herramientas más populares para la prueba de software en Java y ha sido ampliamente adoptado por la comunidad de desarrollo de software.

2.6 Protocolos

2.6.1 HTTP/S

[Hypertext Transfer Protocol \(HTTP\)](#) [37] es un protocolo de comunicación utilizado en la World Wide Web para transferir información entre servidores y clientes. Se utiliza para solicitar y enviar recursos, como páginas web, imágenes y vídeos, a través de la web. [Hypertext Transfer Protocol Secure \(HTTPS\)](#) es una extensión de [HTTP](#), que usa la encriptación para realizar las comunicaciones de manera segura.

2.6.2 REST

[Representational State Transfer \(REST\)](#) [38] es un estilo de arquitectura software para transferir información entre aplicaciones. REST se ha vuelto ampliamente adoptado en el desarrollo de servicios web debido a su simplicidad, flexibilidad y capacidad para aprovechar las capacidades existentes de [HTTP](#), orientado a la comunicación de recursos y sus representaciones.

2.7 Herramientas de desarrollo

2.7.1 IntelliJ

IntelliJ [39] es un [Integrated Development Environment \(IDE\)](#) desarrollado por JetBrains para programar en Java y otros lenguajes de programación. Proporciona herramientas para facilitar la escritura de código, la depuración y la gestión de proyectos. Además, admite una amplia variedad de complementos y herramientas para la integración con otras tecnologías.

2.7.2 Postman

Postman [40] es una herramienta de colaboración y prueba de [API](#) que permite a los desarrolladores probar, documentar y compartir sus [API](#). Permite enviar solicitudes [HTTP](#),

validar respuestas y automatizar tareas relacionadas con la prueba de [API](#). Además, proporciona una interfaz fácil de usar que facilita la creación y gestión de solicitudes.

2.7.3 VSCode

VSCode [\[41\]](#) es un editor de código fuente desarrollado por Microsoft. Ofrece un conjunto de funciones y herramientas para facilitar la programación, como la autocompletación de código, la depuración, el control de versiones, entre otros. VSCode es muy popular entre los desarrolladores debido a su interfaz de usuario intuitiva, su extensibilidad y su amplia gama de características.

2.7.4 DBeaver

DBeaver [\[42\]](#) es una herramienta de gestión de bases de datos multiplataforma que admite una variedad de sistemas de bases de datos como MySQL, PostgreSQL, Oracle, SQL Server y más. Proporciona una interfaz gráfica de usuario intuitiva para realizar operaciones en la base de datos, como consultas, edición de tablas y gestión de conexiones.

2.7.5 GIT

Git [\[43\]](#) es un sistema de control de versiones distribuido, utilizado para registrar cambios en el código fuente durante el proceso de desarrollo del software. Permite a los desarrolladores trabajar de manera colaborativa en un proyecto y realizar un seguimiento de los cambios realizados en el código fuente. Además, proporciona características como control de versiones, control de ramas y restauración de versiones anteriores.

2.7.6 Apache Maven

Apache Maven [\[44\]](#) es una herramienta de gestión de proyectos ampliamente utilizada en el desarrollo de software en Java. Proporciona un enfoque declarativo y basado en configuración para la construcción, gestión de dependencias y documentación de proyectos.

2.7.7 Create React App

Create React App [\[45\]](#) es una herramienta de línea de comandos de Facebook que permite a los desarrolladores crear rápidamente aplicaciones web de React con una configuración predeterminada y optimizada para el desarrollo. Proporciona un entorno de desarrollo rápido y fácil de usar que incluye herramientas como Webpack y Babel para compilar y optimizar el código de la aplicación.

2.8 Sistemas de gestión de base de datos

2.8.1 PostgreSQL

PostgreSQL [46] es un sistema de gestión de bases de datos relacionales de código abierto y muy potente. Ofrece una amplia variedad de características, como soporte para funciones almacenadas, índices avanzados, transacciones y más.

2.8.2 PostGIS

PostGIS [47] es una extensión de PostgreSQL que permite el manejo de datos geoespaciales, como puntos, líneas y polígonos, y ofrece una serie de funciones para el análisis y manipulación de estos datos.

2.9 Tecnologías de alojamiento de servicios

2.9.1 Tomcat

Tomcat [48] es un servidor web y contenedor de servlets de código abierto que implementa las especificaciones de Java Servlet, [JavaServer Pages \(JSP\)](#) y [Java Expression Language \(EL\)](#). Es ampliamente utilizado para ejecutar aplicaciones web desarrolladas en Java y proporciona un entorno de ejecución seguro y escalable para estas aplicaciones.

2.9.2 Webpack

Webpack [49] es una herramienta de empaquetado de módulos de código abierto ampliamente utilizada en el desarrollo web. Su principal objetivo es tomar módulos JavaScript, junto con sus dependencias y otros recursos, y generar paquetes optimizados y eficientes para su uso en el navegador. En concreto en este proyecto se usa Webpack como despliegue en contexto de desarrollo.

2.9.3 Serve

Serve [50] es un servidor web válido para producción, que permite servir contenido estático (páginas [HTML](#), código javascript, [CSS](#) o imágenes) de forma eficiente.

2.9.4 Docker

Docker [51] es una plataforma de contenedorización que permite empaquetar aplicaciones y sus dependencias en contenedores ligeros y portables. En este proyecto la base de datos se despliega mediante contenedores Docker (imagen oficial de PostgreSQL/PostGIS), lo que

facilita la gestión del entorno de desarrollo, la reproducción de la configuración y el despliegue en entornos de integración continua.

Estado del arte

EN ESTE CAPÍTULO se realiza un análisis de las aplicaciones y herramientas existentes que pueden aportar soluciones para abordar la problemática comentada en este documento.

El objetivo no es ofrecer una revisión exhaustiva, sino identificar soluciones relevantes que aporten funcionalidades parecidas a las requeridas: como gestión de avisos/incidencias, representación geoespacial de emergencias y coordinación de equipos en campo.

3.1 Aplicaciones de gestión de incidencias

3.1.1 Jira

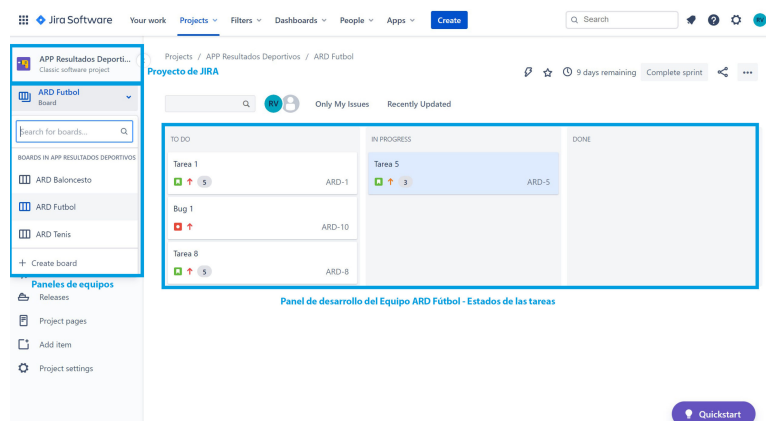


Figura 3.1: Vista de proyecto en Jira

Jira [52] es una plataforma de gestión de incidencias y proyectos que facilita la creación, seguimiento y priorización de tickets. Permite asignar incidencias a usuarios o equipos, definir flujos de trabajo personalizados y enviar notificaciones a través de correo o integraciones con otras plataformas. Es muy usada en entornos de desarrollo de software y en gestión de

servicios.

Una limitación grande para solucionar esta problemática es que no incluye representación geoespacial de incidencias para ello sería necesario integrar componentes de mapas con un sistema GIS.

3.1.2 Asana

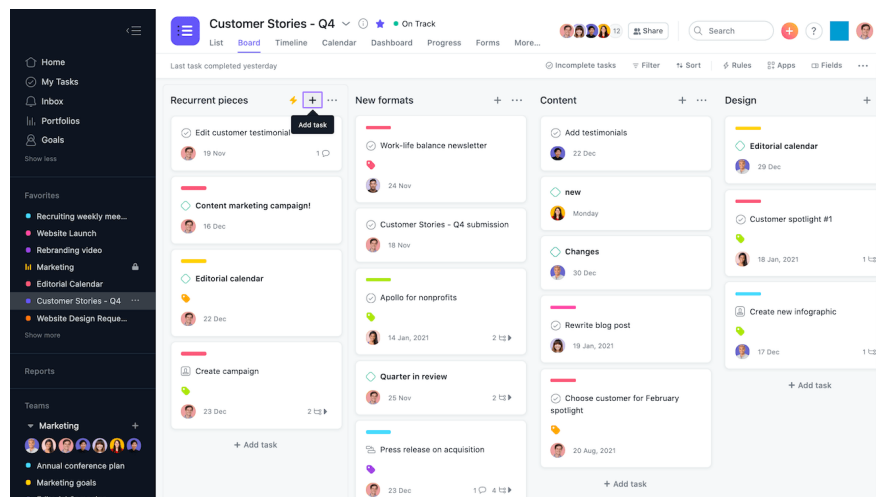


Figura 3.2: Vista de proyecto en Asana

Asana [53] es una herramienta de gestión de tareas y proyectos orientada a la colaboración. Permite la creación de tareas, asignación, prioridades, vistas en lista/kanban/calendario y notificaciones móviles.

Un gran inconveniente es que Asana no dispone de capacidades de representación cartográfica integradas, por lo que su uso en coordinación de emergencias quedaría limitado a la gestión de tareas y comunicaciones.

3.1.3 Línea Verde



Figura 3.3: Vista de la app Línea Verde

Línea Verde [54] es una plataforma de participación ciudadana y gestión de incidencias orientada a ayuntamientos. Ofrece una app gratuita para ciudadanos para notificar problemas y un sistema de gestión para que los equipos municipales asignen, gestionen y cierren las incidencias en tiempo real. También proporciona portales web.

Limitaciones respecto a la problemática de coordinación de emergencias: Línea Verde está claramente orientada al mantenimiento urbano y a la comunicación ciudadano con la administración. Su foco funcional es la gestión de incidencias cotidianas y la mejora del servicio municipal. No parece centrarse en la gestión operativa de emergencias.

3.2 Aplicaciones sobre geolocalización de información

3.2.1 Global Forest Watch - Fires

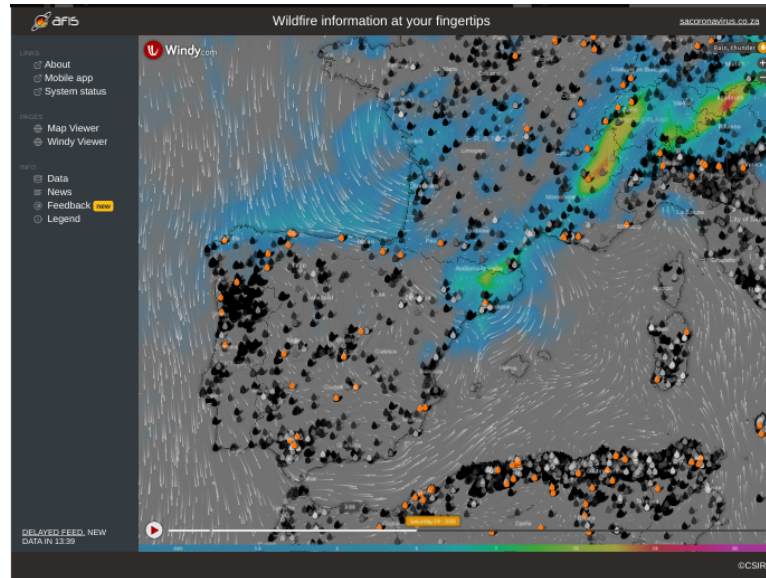


Figura 3.4: Ejemplo de mapa de detección de incendios (AFIS / capa de incendios)

Global Forest Watch Fires [55] ofrece capas de detección de incendios y alertas a partir de datos satelitales. Proporciona herramientas de visualización temporal y filtrado por región.

Un problema a la hora de solventar esta problemática es que se centra en detección remota basada en satélites y no está diseñado para la coordinación operativa de recursos humanos en el terreno ni para la participación ciudadana.

3.2.2 InciWeb

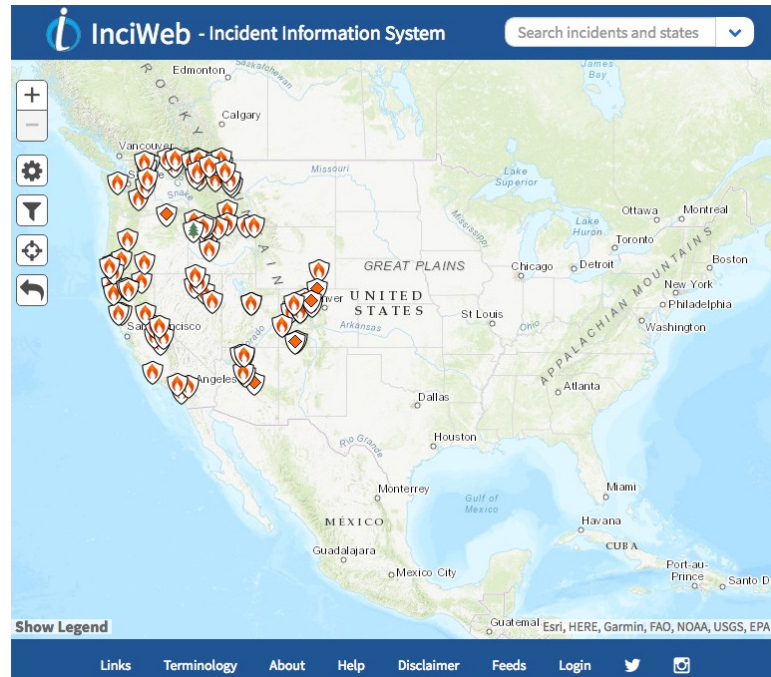


Figura 3.5: Vista de la app InciWeb

InciWeb [56] es una plataforma de información sobre incidentes (especialmente grandes incendios) gestionada por agencias en Estados Unidos. Ofrece mapas, informes y enlaces a recursos locales.

Limitaciones: Está orientada a la difusión de información entre agencias y al público, pero no incluye herramientas para la creación colaborativa de avisos por parte de ciudadanos ni para la gestión táctica de equipos.

3.2.3 Google Maps

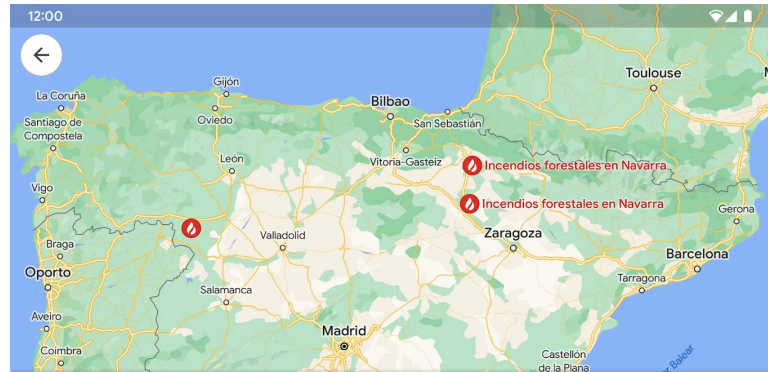


Figura 3.6: Ejemplo de capa/alertas en Google Maps

Google Maps [57] permite mostrar capas y alertas geoespaciales y ofrece APIs fáciles de integrar en aplicaciones. Su ventaja es la amplia base de usuarios y la calidad de su cartografía.

Google Maps no gestiona la parte de despliegue de recursos ni proporciona un flujo de trabajo de incident management; requiere integración con backend para esas funcionalidades.

3.3 Aplicaciones orientadas a organizar grupos de trabajo

3.3.1 Microsoft Teams

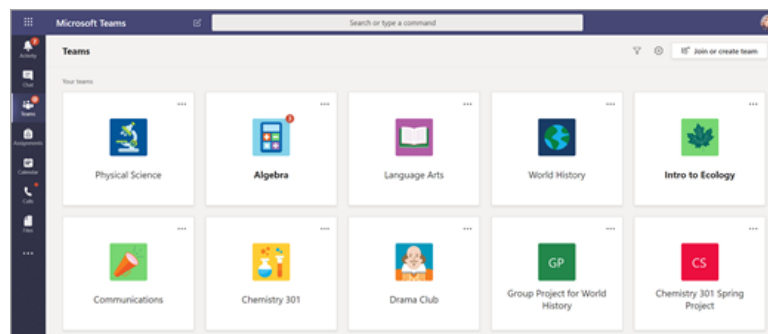


Figura 3.7: Vista equipos en Microsoft Teams

Microsoft Teams [58] es una plataforma de colaboración que combina chat, videollamadas, gestión de archivos y organización por equipos. Es útil para coordinación y comunicación entre miembros de una organización.

Su gran inconveniente es que no ofrece una visión geoespacial de recursos ni integra, por defecto, flujos de notificación orientados a emergencias o despliegue de equipos.

3.3.2 Odoo

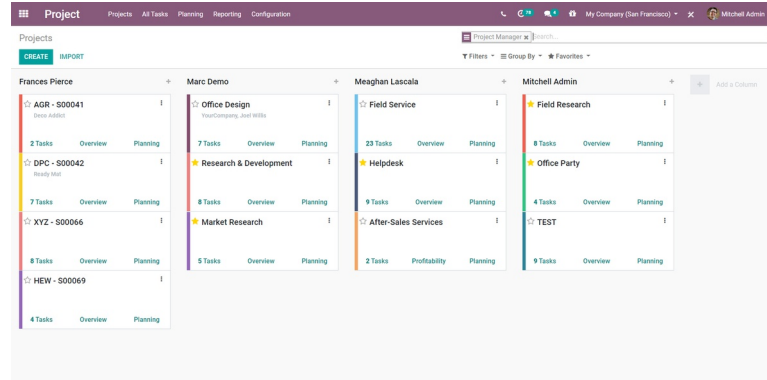


Figura 3.8: Vista equipos en Odoo

Odoo [59] es un ERP modular que puede adaptarse para gestionar proyectos, recursos y comunicaciones. Su flexibilidad permite crear módulos específicos para necesidades concretas.

Para poder utilizar este recurso se requiere desarrollo y personalización que permitan cubrir flujos de coordinación de emergencias; la representación cartográfica suele necesitar módulos adicionales.

3.3.3 PCAM Gestión

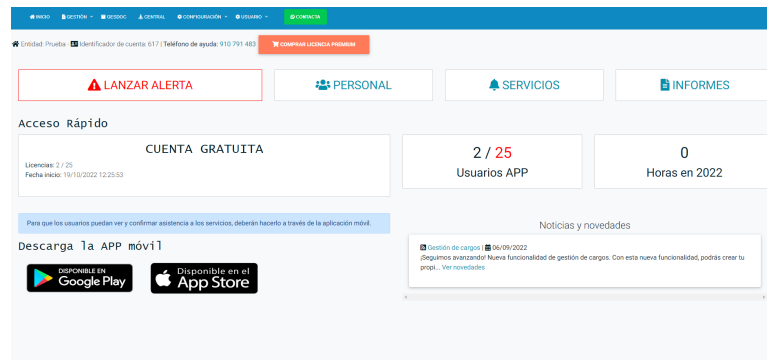


Figura 3.9: Panel de control en PCAM Gestión

PCAM Gestión [60] es una solución orientada a la protección civil que facilita la creación de avisos, asignación de personal y notificaciones. Incluye app móvil y panel de control para administradores.

Esta aplicación integra funcionalidades geoespaciales y visibilidad en tiempo real de recursos pero son limitadas o dependen de módulos adicionales.

3.3.4 XeoCode Lite

XeoCode Lite permite consultar el origen y evolución de incendios, recursos asignados y cartografía asociada. Al ser una herramienta de uso interno normalmente no facilita la participación ciudadana directa.

3.4 Iniciativas públicas recientes

3.4.1 Xunta de Galicia: nueva app y ampliación de detección temprana

En una comunicado, la Xunta de Galicia ha anunciado la ampliación de mecanismos de detección temprana de incendios forestales mediante la creación de una nueva app para la ciudadanía junto el uso de cámaras de videovigilancia.[61].

Esta aplicación combina un canal para notificaciones tempranas, sensores y cámaras para observación remota y procesamiento automatizado con IA para priorizar alertas y detectar anomalías. Todo ello para ayudar a la notificación y coordinación de emergencias en tiempo real.

Funcionalmente esta nueva app parece converger mucho con el dominio funcional del TFG [5]: reportes ciudadanos, integración de sistemas de información dispares como avisos ciudadanos y meteorológicos y también cuenta con representación geoespacial; a efectos prácticos la solución de la Xunta incorpora muchas de las ideas que se prototiparon en el TFG.

Algunas diferencias clave respecto al TFG son que la aplicación añade una capa de hardware y vigilancia (cámaras/sensores) y está diseñada para integrarse en procedimientos administrativos y operativos a escala autonómica. Por otro lado, el TFG aporta un énfasis en la coordinación táctica como la asignación de recursos, gestión por cuadrantes y soporte a decisiones de despliegue en campo.

Como comentario crítico, cabe destacar la coincidencia temática que resalta la relevancia del problema abordado en el TFG y demuestra que las soluciones propuestas tienen aplicación real. Ambas aproximaciones son complementarias y evidencian la necesidad de soluciones integrales que aborden todo el ciclo de gestión de emergencias, desde la detección hasta la coordinación en campo.

3.5 Conclusión

Las aplicaciones analizadas ofrecen soluciones parciales en las tres dimensiones de interés: gestión de avisos, representación geoespacial y coordinación de equipos. Ninguna de las herramientas revisadas integra de forma nativa las tres capacidades con enfoque ciudadano-operativo y

despliegue de recursos; esto evidencia el hueco funcional que aborda la propuesta de este trabajo.

La tabla comparativa 3.1 resume las funcionalidades analizadas para resaltar fortalezas y carencias de cada alternativa:

	Visualización geográfica	Datos meteorológicos	Gestión de personal y recursos	Plan gratuito suficiente	Plugins o margen de mejora	Sistema de avisos o notificaciones	Multi-emergencia	App móvil
<i>Jira</i>			✓	✓	✓	✓	✓	✓
<i>Asana</i>			✓	✓	✓	✓	✓	✓
<i>Línea Verde</i>	✓		✓	✓	✓	✓	✓	✓
<i>Global Forest Watch - Fires</i>	✓	✓		✓		✓		
<i>InciWeb</i>	✓			✓				
<i>Google Maps</i>	✓			✓	✓			✓
<i>Microsoft Teams</i>			✓	✓	✓	✓	✓	✓
<i>Odoo</i>			✓	✓	✓	✓	✓	✓
<i>PCAM Gestión</i>	✓		✓		✓	✓	✓	✓
<i>XeoCode Lite</i>	✓	?	✓	?	?	?	?	?
<i>Xunta (nueva app)</i>	✓	✓		✓		✓		✓

Tabla 3.1: Tabla de comparativa entre aplicaciones

Análisis de viabilidad

EN ESTE CAPÍTULO se analiza la viabilidad del proyecto desde tres perspectivas complementarias: dimensión temporal, dimensión tecnológica y dimensión económica.

4.1 Viabilidad temporal

Para la realización de este **TFM** se ha estimado una duración aproximada de cuatro meses. Dicha planificación incluye las fases de análisis, diseño, implementación, pruebas y redacción de la memoria. Además, se contempla la familiarización de las nuevas tecnologías. Dado el alcance definido la viabilidad temporal es aceptable y no representa un riesgo crítico para la finalización del proyecto.

La metodología usada utiliza desarrollos cortos para ir validando el progreso y ajustando el alcance si fuera necesario.

4.2 Viabilidad tecnológica

Para el estudio de viabilidad tecnológica se han considerado las tecnologías heredadas del **TFG**, así como las nuevas tecnologías propuestas para el desarrollo de la aplicación móvil y la integración de notificaciones push.

En resumen, el stack tecnológico propuesto para el **TFM** es el siguiente:

- Backend: Spring Boot (Java) para servicios REST y la integración con PostgreSQL/PostGIS.
- Base de datos: PostgreSQL con PostGIS para soporte espacial.
- Frontend web: React con Material-UI para interfaces ricas.
- Aplicación Android: Kotlin y Jetpack Compose para la capa de presentación; cliente nativo para acceder a sensores y recibir notificaciones push.

- Notificaciones push: Firebase Cloud Messaging (FCM) para la entrega fiable de mensajes a dispositivos Android.

En lo relativo de Kotlin y FCM, ambas tecnologías están ampliamente adoptadas, cuentan con extensa documentación, y existen guías oficiales para su integración con backends Java/Spring. Por tanto, su adopción no supone un riesgo tecnológico significativo; aportan ventajas en productividad, interoperabilidad móvil y soporte nativo para notificaciones.

Las demás tecnologías son conocidas por el autor gracias a su dedicación profesional y al proyecto previo, lo que reduce el riesgo de adopción y acelera el desarrollo.

4.3 Viabilidad económica

El proyecto no requiere una inversión económica significativa en hardware o licencias de software. El software utilizado es mayoritariamente Open Source (Spring Boot, PostgreSQL, React, Kotlin/Jetpack Compose), por lo que no hay costes de licencia en el entorno de desarrollo académico.

En el ámbito del desarrollo y las pruebas estos pueden ejecutarse con recursos disponibles por lo que no es necesario adquirir nuevo hardware. Sobre el uso de servicios en la nube o APIs de terceros (FCM, MapLibre, OpenWeatherMap) pueden surgir costes, pero se pueden controlar gracias a los planes gratuitos o a la monitorización de uso.

La dimensión económica no supone un riesgo significativo para la ejecución del TFM pues los costes directos son bajos.

4.4 Conclusión del análisis de viabilidad

En resumen, el proyecto es totalmente viable porque combina un plan de trabajo realista de cuatro meses con un uso inteligente de la tecnología. Al reutilizar la base del TFG y apoyarse en herramientas gratuitas y potentes como Kotlin y FCM, el riesgo técnico es mínimo y el coste es prácticamente cero. Esta experiencia previa, sumada a una planificación clara, garantiza que el proyecto se pueda terminar a tiempo y con la calidad necesaria para un TFM.

Introducción al desarrollo realizado

5.1 Introducción

EN este capítulo se presenta una descripción del desarrollo realizado, los objetivos perseguidos y la metodología aplicada durante la implementación. El propósito es ofrecer un vistazo general de los artefactos construidos y de las decisiones más relevantes tomadas durante el desarrollo del proyecto.

5.2 Arquitectura de la Solución

PENDIENTE una vez avanzado más el proyecto

5.3 Metodología e iteraciones

El proceso de desarrollo se organizó mediante iteraciones cortas con objetivos funcionales concretos, siguiendo las ideas de Scrum adaptadas al contexto de trabajo individual. Aunque no se aplicó una implementación completa y formal de Scrum (debido al tamaño reducido del equipo), se adoptaron sus pilares fundamentales: planificación por sprints, revisiones y *feedback* continuo.

En cuanto a roles, el autor del TFM asumió las responsabilidades de desarrollo y encargado de definición funcional, mientras que el tutor actuó como responsable de supervisión y de aceptación.

Se establecieron reuniones de revisión mensuales con el tutor en las que se presentaron los artefactos construidos (prototipos, endpoints, pantallas, pruebas y documentación). Estas sesiones sirvieron para validar funcionalidades, priorizar tareas y ajustar el dominio del TFM y el alcance de las siguientes iteraciones.

El proyecto se estructuró en una serie de sprints. De una forma resumida, las fases necesarias para realizar este TFM fueron: una fase inicial de contextualización y definición del valor del proyecto; la creación del esqueleto y la arquitectura base de la aplicación móvil para facilitar el desarrollo posterior; la normalización y actualización del modelo de recursos y organizaciones manteniendo compatibilidad con las entidades Equipo y Vehículo; la implementación de la gestión de emergencias y asignaciones básicas; la introducción de un esquema de logs operativos y la migración o compatibilidad con tablas legacy; la integración de notificaciones móviles mediante FCM y el flujo de aceptación de asignaciones; y, finalmente, la implementación de capacidades semiautomáticas junto a un motor de reglas sencillo para generar propuestas automáticamente.

Para la planificación y el detalle de cada sprint se describen con mayor extensión en el capítulo de planificación (capítulo 6.2).

Planificación y análisis de costes

EN este capítulo se presenta la planificación aplicada durante el desarrollo del proyecto y un análisis de los costes asociados. Se describe la planificación inicial, las sprints que han estructurado el trabajo y las desviaciones observadas en el seguimiento.

6.1 Planificación

La planificación inicial se definió en 9 sprints, cada una organizada internamente siguiendo el modelo clásico de cuatro fases: análisis, diseño, implementación y pruebas. La estimación de esfuerzo inicial se basó en jornadas de 6 horas diarias y en una duración prevista del proyecto de cuatro meses. No obstante, las complejidades técnicas y errores imprevistos motivaron desviaciones en varias iteraciones, tal y como se comenta en la sección de seguimiento.

Tarea	Fecha inicio	Días estimados	Días reales
Definición tipos Emergency	04/03	0.5	0.5
Definición modelo Assignment	04/03	0.5	0.5
Definición contratos OpenApi	04/03	1	1
Diseño entidades JPA	05/03	0.5	0.5
Implementación entidad Emergency	06/03	0.5	0.5
Implementación entidad Assignment	06/03	0.5	0.5
Implementación repositorios	06/03	0.5	0.5
UseCase Emergency	08/03	0.5	0.5
UseCase Assignment	08/03	0.5	0.5
Endpoints /emergencies	09/03	0.5	0.5
Endpoints /assignments	09/03	0.5	0.5
Tests unitarios backend	11/03	1	1
Tests integración backend	11/03	1	1
Consumir Api desde frontend	10/03	0.5	0.5
Vista mapa (crear Emergency)	11/03	1	1
Formulario Emergency	11/03	0.5	0.5
Listado Emergencies	12/03	0.5	0.5
Actualización UI Assignment	12/03	1	1
Listado Assignments	13/03	0.5	0.5
Consumir Api desde Android	07/03	0.5	0.5
Lista Assignments Android	12/03	1	1
Detalle Assignment Android	13/03	0.5	0.5
Corrección bugs	15/03	1	1

Tabla 6.1: Planificación del Sprint 3: Emergencias y asignaciones básicas

En la tabla 6.1 se puede observar un desglose del sprint 3, que se centró en la implementación de la gestión de emergencias y asignaciones básicas. Cada tarea se estimó inicialmente en función de su complejidad, esto sirvió para poder observar desviaciones y ajustar la planificación de los siguientes sprints. En este caso, el sprint se completó dentro del plazo previsto, lo que permitió mantener el ritmo de desarrollo planificado.

6.2 Sprints

El proyecto se estructuró en una sucesión de sprints, cada uno de los cuales incrementaba la funcionalidad de los artefactos entregados. A continuación se detalla cada sprint con sus objetivos y resultados principales.

6.2.1 Sprint 0: Contextualización

En esta fase inicial se realizó la contextualización del dominio y la justificación del valor del TFM. Se investigaron fuentes, alternativas a la aplicación y tecnologías potenciales (mapas, sistema de notificaciones android, sistema de recomendación y reglas básico). El objetivo fue delimitar el alcance y dominio del proyecto para que este sea viable en un espacio temporal acotado.

6.2.2 Sprint 1: Esqueleto y arquitectura base de aplicación móvil

Se creó el esqueleto del proyecto móvil y se definieron las dependencias y la arquitectura base (estructura de paquetes, patrones de navegación, y la configuración inicial de Jetpack Compose). Además se implementaron los módulos iniciales de autenticación y sincronización con el Backend (endpoints básicos) para poder iterar sobre funcionalidades reales.

6.2.3 Sprint 2: Actualizar la organización de recursos

Se introdujo el modelo unificado de recursos y organizaciones preservando la compatibilidad con las entidades Legacy (Equipo y Vehículo). Esto incluyó definir atributos de capacidad, áreas operativas (cuadrantes) y las relaciones necesarias a nivel de base de datos y DTOs. Se arreglaron los endpoints antiguos y DTO's parara la gestión CRUD de organizaciones y sus recursos.

6.2.4 Sprint 3: Emergencias y asignaciones básicas

Se implementó la gestión de emergencias con soporte para dos tipos de localización: punto y cuadrante. Se añadió la creación y edición de Emergencias, la definición de cuadrantes y una primera versión del flujo de creación de Asignaciones. Esta iteración incluyó validaciones de consistencia (ej. comprobar que un cuadrante pertenece a una emergencia) y pruebas de integración explícitas para el servicio de Asignaciones.

6.2.5 Sprint 4: Nuevo sistema de logs operativos

Se definió un nuevo esquema para los logs operativos, ahora centralizados en las Asignaciones de recursos en las emergencias. El trabajo abarcó el nuevo modelado y la persistencia que

permiten conservar el histórico.

6.2.6 Sprint 5: Notificaciones en Android y aceptación de asignaciones

Se integró Firebase Cloud Messaging (FCM) en la aplicación Android: registro de MobileDevice, manejo de tokens y recepción de notificaciones. Se implementó el flujo de envío de notificaciones desde el Backend y la experiencia de usuario en la app para aceptar o rechazar asignaciones recibidas.

6.2.7 Sprint 6: Protocolos semiautomáticos (motor de reglas simple)

Se introdujo la capacidad de definir Protocols almacenados en JSONB y un motor de reglas ligero que permite generar propuestas de Asignaciones de forma semiautomática en el frontend. Esto incluyó la definición del formato JSON para los protocolos, la implementación de un evaluador simple y la integración con el pipeline de creación de propuestas.

6.2.8 Sprint 7 y 8: Finalización, pruebas y despliegue

Las iteraciones finales se dedicaron a la estabilización: pruebas end-to-end, ajuste de UX tanto del frontal como de la aplicación Android, corrección de errores, poblamiento de datos de ejemplo y preparación de artefactos de despliegue (contenedores Docker para BBDD y servicio). También incluyeron la redacción y revisión de la memoria y la generación de la documentación.

6.3 Seguimiento

El seguimiento se llevó a cabo mediante reuniones mensuales con el tutor y el uso de un tablero Kanban de tareas que permitía visualizar el estado de ellas. En cada revisión se presentaron artefactos funcionales (pantallas, endpoints, wireframes).

Las desviaciones más relevantes provinieron de problemas técnicos con la representación de cuadrantes en el mapa, configuración de la aplicación Android y el motor de reglas básico para los protocolos.

6.4 Análisis de costes

El análisis de costes se ha planteado distinguiendo entre costes directos de personal, costes indirectos de infraestructura y un margen de contingencia para imprevistos. Para la valoración económica del esfuerzo se han tomado como referencia salarios publicados por Talent.com para España, donde un perfil de *Programador* se sitúa en 27.500 €/año (14,10 €/h) [62]y

un perfil de *Analista programador* en 29.120 €/año (14,93 €/h) [63]. Estos valores resultan razonables para un TFM técnico con tareas de análisis, desarrollo, pruebas e integración.

6.4.1 Estimación del esfuerzo

La planificación se ha estructurado en 7 sprints. Si se asume una duración media de 2 semanas por sprint, el proyecto cubre aproximadamente 14 semanas de trabajo efectivo. Considerando una dedicación máxima de 6 horas al día y trabajo únicamente de lunes a viernes, el esfuerzo total estimado se calcula como sigue:

$$7 \text{ sprints} \times 2 \text{ semanas/sprint} \times 5 \text{ días/semana} \times 6 \text{ h/día} = 420 \text{ horas}$$

Este valor representa una estimación prudente del esfuerzo total del proyecto. La distribución del tiempo no es homogénea entre sprints, ya que las primeras iteraciones requieren más análisis y diseño, mientras que las finales concentran más trabajo de integración, pruebas, corrección de errores y documentación.

6.4.2 Costes directos de personal

Para la valoración del trabajo se ha tomado como base el perfil de *Analista programador*, al ser el que mejor refleja un proyecto de estas características, donde una misma persona asume tareas de definición funcional, implementación y validación. Con la tarifa media indicada por la fuente utilizada, el coste directo estimado es:

$$420 \text{ h} \times 14,93 \text{ €/h} = 6.270,60 \text{ €}$$

De forma complementaria, si se valorase el trabajo con el perfil de *Programador*, el coste sería:

$$420 \text{ h} \times 14,10 \text{ €/h} = 5.922,00 \text{ €}$$

Para reflejar ambas referencias, se adopta como coste base del proyecto el punto medio entre ambos perfiles, es decir, **6.096,30 €**, que representa mejor la mezcla de tareas de programación y análisis realizada durante el desarrollo.

6.4.3 Costes indirectos

A los costes de personal se añaden los costes indirectos de ejecución. Para este tipo de proyecto, los más relevantes son la amortización del equipo de trabajo y los suministros básicos asociados al desarrollo:

- Amortización de hardware: se considera el uso de un equipo portátil de desarrollo valorado en 900 €, con una vida útil de 4 años. Para un proyecto de unos 3,5 meses, el coste imputable es de aproximadamente 66,00 €.
- Conectividad y suministros: se estiman 40 €/mes de internet y 30 €/mes de electricidad proporcional al tiempo de desarrollo. Para 3,5 meses, esto supone 245,00 €.

6.4.4 Resumen económico

Concepto	Importe estimado (€)	Observaciones
Coste directo de personal	6.096,30	Media entre programador y analista programador
Amortización de hardware	66,00	Portátil de desarrollo imputado al proyecto
Conectividad y suministros	245,00	Internet y electricidad proporcionales
Total estimado	6.407,3	

Tabla 6.2: Desglose de costes estimados del proyecto

La tabla 6.2 muestra que el mayor peso del presupuesto corresponde al trabajo humano, seguido por los costes de infraestructura y suministros. Esto es coherente con un proyecto de software de ámbito académico, donde la inversión principal no está en equipamiento especializado sino en la dedicación de horas de desarrollo, integración y validación.

6.4.5 Conclusión del análisis de costes

El coste final del proyecto asciende a **6.407,3€** con una dedicación total de **420 horas**. Estas cifras reflejan el alcance y la coste del TFM, considerando tanto el esfuerzo humano como los recursos materiales necesarios para su ejecución. Es importante destacar que esta estimación se ha realizado con criterios prudentes y basándose en datos salariales reales, lo que aporta una visión más ajustada del valor económico del trabajo realizado en este proyecto.

Requisitos del sistema

X^{xxx,...}

7.1 Introducción

Aspectos a considerar

- Transcripción textual/completa de los requisitos, punto de partida para identificar actores y a continuación casos de uso (siguientes secciones).
- Pueden diferenciarse entre requisitos funcionales y no funcionales

7.2 Actores

7.3 Casos de uso

Aspectos a considerar

- Agrupar casos de uso (servirá de base para agruparlos en servicios posteriormente en la sección de diseño).
- Definición concreta /detallada de cada caso de uso (precondiciones ...)

7.4 Modelo de casos de uso

Aspectos a considerar

- Diagrama de casos de uso incluyendo actores y relaciones entre casos

7.5 Prototipado de interfaz

Es interesante hacer el esfuerzo en este momento de prototipar las principales pantallas de la interfaz (y añadirlas en la memoria), como parte del análisis de requisitos, así como de aspectos de usabilidad de las mismas.

X^{xxx,...}

8.1 Introducción y objetivos

8.2 Resumen de patrones usados

8.3 Arquitectura general

Aspectos a considerar

- Arquitectura aplicación SPA
- División en subsistemas: En una aplicación SPA al menos backend y frontend pero podría haber más si se decide implementar utilidades e.g. como módulos independientes (e.g. como proyectos maven / jars independientes)

8.4 Subsistema Backend

8.4.1 Objetivos

8.4.2 Arquitectura

Aspectos a considerar

- Diagrama de paquetes general del subsistema (figura y descripción de la misma)

8.4.3 Modelo del dominio

Diagrama de entidades

Aspectos a considerar

- Diagrama de clases persistentes de la aplicación
- Para cada indicar, tanto de su semántica como de sus atributos.
- Justificar decisiones de diseño (¿desnormalizaciones?, métodos de lógica - Domain Model Pattern - ?)

Modelo de datos

Aspectos a considerar

- Diagrama entidad-relación
- Diccionario de Datos (en su ausencia - por temas de espacio bastante razonable - compensable con defición clara en diagrama de entidades).

Consideraciones adicionales de modelado de datos

Aspectos a considerar

- Claves primarias autogeneradas
- Anotaciones para relaciones entre entidades
- Optimistic Locking y @Version en Hibernate
- Relaciones LAZY/EAGER
- Optimizaciones hibernate en navegación entre entidades
- Optimizaciones hibernate - cache primer nivel

8.4.4 Capa de acceso a datos

Aspectos a considerar

- Arquitectura de un DAO
- Aspectos más relevantes de los DAOs implementados

8.4.5 Capa de lógica de negocio

Aspectos a considerar

- Arquitectura de servicios del modelo: Interfaces + clases asociadas
- Incluir descripción de métodos (en algunos casos podrían comentarse de forma agregada algunas operaciones por ejemplo gestión de usuarios (añadir/eliminar/modificar) en lugar de comentar de uno en uno.
- Incluir descripción de algoritmos en los casos que sea necesario. Podría incluirse un diagrama de secuencia para complementar algún algoritmo.

8.4.6 Capa servicios

Aspectos a considerar

- Diseño controladores
- API REST (intentar que sea REST-full cuando haya un mapping directo ... overloading del POST en otro caso
- Gestión de errores y aspectos de internacionalización de mensajes de error
- Autenticación y aspectos de control de acceso a los diferentes endpoints (SecurityConfig). Soporte de redirección tras autenticación, si implementado.

8.4.7 Otros?

Aspectos a resaltar

- Integración con otros sistemas o librerías a nivel de backend, e.g. pago con PayPal.
- Tb se podrían destacar aspectos como chats con websockets o algoritmos. Importante dejar claro todo lo que se comente en "objetivos" y "aportaciones" principales del proyecto.

8.5 Subsistema Frontend

8.5.1 Objetivos

8.5.2 Arquitectura

Aspectos a considerar

- Arquitectura de capas: al menos capa de acceso al servicio y capa de componentes de interfaz

8.5.3 Capa de acceso al servicio**8.5.4 Capa de componentes de interfaz****8.5.5 Otros?****Aspectos a considerar**

- Internacionalización de mensajes de texto
- Integración con otros sistemas / librerías

Implementación

X^{xxx,...}

9.1 Software requerido

Pre-requisitos software para poder compilar el código fuente. Respecto a la base de datos, puede referenciarse al apéndice de instrucciones de instalación, a la parte de instalación y configuración de la base de datos.

9.2 Estructura

Estructura de carpetas del código fuente. Si se utiliza maven, básicamente indicar estructura estándar de maven + casos especiales como localización de fichero de creación de datos ...

9.3 Instrucciones de compilación

Instrucciones de cómo compilar el código fuente. Si se usa maven, básicamente comentar las fases principales a usar de maven.

Comentar también cómo ejecutar en desarrollo.

Capítulo 10

Pruebas

X^{xxx,...}

10.1 Introducción

Aspectos a considerar

- Importancia de las pruebas, tipos de pruebas realizadas en este proyecto y metodología de las pruebas.

10.2 Pruebas unitarias

10.3 Pruebas de integración

Aspectos a considerar

- Importante indicar si son automatizadas o manuales.
- En el caso de pruebas del modelo, casi seguro que serán automatizadas. Necesario indicar la idea de las pruebas, repetibles para detectar posibles problemas de regresión en el futuro e independientes unas de otras: cada prueba crea lo necesario para validar el caso de uso, invoca el caso de uso a probar y valida los efectos asociados a la invocación del caso de uso y eliminar todos los datos generados. Importante validar diferentes escenarios de éxito y fallo posibles para cada caso de uso.
- En las pruebas de integración, Recomendable incluir el nivel de cobertura de las mismas(existen plugins que lo calculan de forma sencilla).
- En el caso de pruebas de capa web, si no se han automatizado será necesario describir el plan de pruebas.

10.4 ...

Conclusiones y futuras líneas de trabajo

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras, (no caso de [TFM](#), carácter profesional del mesmo)...

11.1 Conclusiones

11.2 Aprendizajes

11.3 Futuras líneas de trabajo

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

A.1 Instalación del Software

A.2 Manual de usuario

Introducción al layout general de la aplicación, y subapartados para cada una de las secciones principales, para que sea más sencillo de consultar / comprender.

Lista de acrónimos

API Application Programming Interface. [7](#), [8](#), [10](#), [11](#)

AXEGA Axencia Galega de Emerxencias (112 Galicia). [1](#), [2](#)

Compose Jetpack Compose. [9](#)

CSS Cascading Style Sheet. [5](#), [6](#), [12](#)

EL Java Expression Language. [12](#)

FCM Firebase Cloud Messaging. [3](#), [9](#), [24](#), [26](#), [30](#)

FMS FirebaseMessagingService. [9](#)

HTML HyperText Markup Language. [5](#), [6](#), [12](#)

HTTP Hypertext Transfer Protocol. [10](#)

HTTPS Hypertext Transfer Protocol Secure. [10](#)

IDE Integrated Development Environment. [10](#)

IoC Inversión de Control. [7](#)

JDBC Java Database Connectivity. [7](#)

JPA Java Persistence API. [7](#)

JSON JavaScript Object Notation. [6](#)

JSONB JavaScript Object Notation Binary. [3](#), [30](#)

JSP JavaServer Pages. [12](#)

Material Material Design. 9

PLADIGA Plan de Prevención y Defensa Contra los Incendios Forestales de Galicia. 2

REST Representational State Transfer. 10

SQL Structured Query Language. 5, 7

TFG Trabajo Fin de Grado. 1–3, 21, 23, 24

TFM Trabajo Fin de Máster. 1–3, 23–26, 29, 32, 42

Bibliografía

- [1] M. para la Transición Ecológica y el Reto Demográfico (MITECO), “Estadística general de incendios forestales (egif),” 2025, consultado el 2026-05-07. [En línea]. Disponible en: <https://www.miteco.gob.es/es/biodiversidad/temas/incendios-forestales/estadisticas-datos.html>
- [2] A. G. de Emerxencias (AXEGA), “O 112 galicia xestionou unha decena de incidencias relacionadas coa chuvia en ourense, a rúa e o carballiño,” 2026, consultado el 2026-05-07. [En línea]. Disponible en: <https://www.axega112.gal/gl/content/o-112-galicia-xestionou-unha-decena-de-incidencias-relacionadas-coa-chuvia-en-ourense-rua-e>
- [3] —, “Alerta laranxa por chuvias intensas no noroeste de ourense,” 2026, consultado el 2026-05-07. [En línea]. Disponible en: https://www.axega112.gal/gl/alertas_activas
- [4] —, “Rexistrado un sismo de magnitude 2.6 con epicentro en ponteceso,” 2026, consultado el 2026-05-07. [En línea]. Disponible en: <https://www.axega112.gal/gl/content/rexistrado-un-sismo-de-magnitude-26-con-epicentro-en-ponteceso>
- [5] A. García Vázquez, “Aplicación para coordinación de equipos para la prevención y extinción de incendios forestales,” Memoria de TFG, Universidade da Coruña, 2023. [En línea]. Disponible en: <https://ruc.udc.es/entities/publication/04b8901f-0960-4d07-8cce-9a2ecfcb471>
- [6] A. G. de Emerxencias (AXEGA), “Portal oficial de axega e 112 galicia,” 2026, consultado el 2026-05-07. [En línea]. Disponible en: <https://www.axega112.gal>
- [7] X. e. D. X. d. G. Consellería de Presidencia, “Plans de emerxencia de galicia,” 2026, consultado el 2026-05-07. [En línea]. Disponible en: <https://cpxt.xunta.gal/plans-de-emerxencia>
- [8] C. do Medio Rural e Desenvolvemento Sostible. Xunta de Galicia, “Pladiga: Plan de prevención e defensa contra os incendios forestais (pladiga),” 2026, consultado el 2026-

- 05-07. [En línea]. Disponible en: <https://mediorural.xunta.gal/es/temas/defensa-monte/pladiga-2026>
- [9] J. del Estado, “Ley 17/2015, de 9 de julio, del sistema nacional de protección civil,” 2015, consultado el 2026-05-07. [En línea]. Disponible en: <https://www.boe.es/eli/es/l/2015/07/09/17/con>
- [10] M. del Interior, “Real decreto 524/2023, de 20 de junio, por el que se aprueba la norma básica de protección civil,” 2023, consultado el 2026-05-07. [En línea]. Disponible en: <https://www.boe.es/eli/es/rd/2023/06/20/524/con>
- [11] Oracle, “Java,” <https://www.oracle.com/java/>, consultado el 2026-05-07.
- [12] JetBrains, “Kotlin programming language,” <https://kotlinlang.org/>, consultado el 2026-05-07.
- [13] W3Schools, “Sql,” <https://www.w3schools.com/sql/>, consultado el 2026-05-07.
- [14] M. W. Docs, “Html y css,” https://developer.mozilla.org/es/docs/Learn/Getting_started_with_the_web/HTML_basics, consultado el 2026-05-07.
- [15] Mozilla, “Javascript,” <https://developer.mozilla.org/es/docs/Web/JavaScript>, consultado el 2026-05-07.
- [16] “Jsx (javascript xml),” <https://reactjs.org/docs/introducing-jsx.html>, accedido el 2026-05-07.
- [17] “Latex,” <https://www.latex-project.org/help/documentation/>, accedido el 2026-05-07.
- [18] “Json (javascript object notation),” <https://www.json.org/>, accedido el 2026-05-07.
- [19] Spring, “Spring,” <https://spring.io/>, accedido el 2026-05-07.
- [20] —, “Spring boot,” <https://spring.io/projects/spring-boot>, consultado el 2026-05-07.
- [21] —, “Spring data,” <https://spring.io/projects/spring-data>, consultado el 2026-05-07.
- [22] Oracle, “Java persistence api,” <https://docs.oracle.com/javaee/6/tutorial/doc/bnbpz.html>, consultado el 2026-05-07.
- [23] Hibernate, “Hibernate orm,” <https://hibernate.org/orm/>, consultado el 2026-05-07.
- [24] JDBC, “Jdbc,” <https://www.ibm.com/docs/es/developer-for-zos/9.5.1?topic=support-what-is-jdbc>, consultado el 2026-05-07.
- [25] Facebook, “React,” <https://reactjs.org/>, consultado el 2026-05-07.

- [26] Redux, “Redux,” <https://redux.js.org/>, consultado el 2026-05-07.
- [27] R. Toolkit, “Redux toolkit query,” <https://redux-toolkit.js.org/rtk-query/overview>, consultado el 2026-05-07.
- [28] Material-UI, “Material-ui,” <https://material-ui.com/>, consultado el 2026-05-07.
- [29] O. W. Map, “Open weather map,” <https://openweathermap.org/>, consultado el 2026-05-07.
- [30] M. Community, “Maplibre,” <https://maplibre.org/>, accedido el 2026-05-07.
- [31] G. Inc., “Gradle build tool,” <https://gradle.org/>, consultado el 2026-05-07.
- [32] A. Developers, “Jetpack compose,” <https://developer.android.com/jetpack/compose>, consultado el 2026-05-07.
- [33] Google, “Material design,” <https://material.io/>, consultado el 2026-05-07.
- [34] G. Firebase, “Firebase cloud messaging (fcm),” <https://firebase.google.com/docs/cloud-messaging>, consultado el 2026-05-07.
- [35] A. D. . Firebase, “Firebasemessaging (android),” <https://firebase.google.com/docs/cloud-messaging/android/receive>, consultado el 2026-05-07.
- [36] JUnit.org, “Junit,” <https://junit.org/junit5/>, consultado el 2026-05-07.
- [37] Mozilla, “Http,” <https://developer.mozilla.org/es/docs/Web/HTTP>, consultado el 2026-05-07.
- [38] REST, “Rest,” <https://restfulapi.net/>, accedido el 2026-05-07.
- [39] JetBrains, “Intellij,” <https://www.jetbrains.com/idea/>, consultado el 2026-05-07.
- [40] Postman, “Postman,” <https://www.postman.com/>, consultado el 2026-05-07.
- [41] Microsoft, “Vscode,” <https://code.visualstudio.com/>, consultado el 2026-05-07.
- [42] DBeaver, “Dbeaver,” <https://dbeaver.io/>, consultado el 2026-05-07.
- [43] Git, “Git,” <https://git-scm.com/>, consultado el 2026-05-07.
- [44] “Apache maven,” <https://maven.apache.org/>, Apache Software Foundation, accedido el 2026-05-07.
- [45] Facebook, “Create react app,” <https://create-react-app.dev/>, consultado el 2026-05-07.

- [46] P. G. D. Group, "Postgresql," <https://www.postgresql.org/>, consultado el 2026-05-07.
- [47] P. D. Team, "Postgis," <https://postgis.net/>, consultado el 2026-05-07.
- [48] T. A. S. Foundation, "Apache tomcat," <https://tomcat.apache.org/>, consultado el 2026-05-07.
- [49] O. S. Community, "Webpack," <https://webpack.js.org/>, consultado el 2026-05-07.
- [50] Serve, "Serve," <https://www.npmjs.com/package/serve?activeTab=readme>, consultado el 2026-05-07.
- [51] I. Docker, "Docker," <https://www.docker.com/>, consultado el 2026-05-07.
- [52] Atlassian, "Jira software," <https://www.atlassian.com/software/jira>, consultado el 2026-05-07.
- [53] I. Asana, "Asana," <https://asana.com/>, consultado el 2026-05-07.
- [54] L. V. Municipal, "Línea verde municipal - plataforma de participación ciudadana y gestión de incidencias," <https://www.lineaverdemunicipal.info/>, consultado el 2026-05-07.
- [55] G. F. Watch, "Global forest watch fires," <https://www.globalforestwatch.org/>, consultado el 2026-05-07.
- [56] U. W. F. Information, "Inciweb - incident information system," <https://inciweb.nwgc.gov/>, consultado el 2026-05-07.
- [57] Google, "Google maps platform," <https://maps.google.com/>, consultado el 2026-05-07.
- [58] Microsoft, "Microsoft teams," <https://www.microsoft.com/en/microsoft-teams/group-chat-software>, consultado el 2026-05-07.
- [59] O. S.A., "Odoo," <https://www.odoo.com/>, consultado el 2026-05-07.
- [60] PCAM, "Pcam gestión," <https://www.pcamexample.org/>, consultado el 2026-05-07.
- [61] X. de Galicia, "La xunta amplía los mecanismos de detección temprana de incendios forestales con una nueva app para la ciudadanía," <https://www.xunta.gal/es/notas-de-prensa/-/nova/022747/xunta-amplia-los-mecanismos-deteccion-temprana-incendios-forestales-con-una-nueva>, 2026, consultado el 2026-05-07.

- [62] Talent.com, “Salario de un programador en españa,” <https://es.talent.com/salary?job=programador>, consultado el 2026-05-07.
- [63] —, “Salario de un analista programador en españa,” <https://es.talent.com/salary?job=analista+programador>, consultado el 2026-05-07.